

1. Wait/Die Scheme

Timestamps

T1=101, T2=102, T3=103, T4=104, T5=105.

T3 R(X)	T3 requests and is granted a read-lock on X ✓
T5 R(X)	T5 requests and is granted a read-lock on X ✓
T1 W(X)	T1 requests an exclusive lock on X, T3 and T5 hold a read-lock on X, T1 is older than both T3 and T5, T1 waits for T3 and T5 to complete ✓
T2 W(Y)	T2 requests and is granted an exclusive lock on Y ✓
T3 R(Y)	T3 requests a read-lock on Y, T2 holds an exclusive lock on Y, T3 is younger than T2, T3 dies/ rolls back and is rescheduled with the original timestamp ✓
T5 W(Z)	T5 requests and is granted an exclusive lock on Z ✓
T5 COMMITS	T5 releases all locks ✓ Which locks? And what else happens?
T2 R(X)	T2 requests and is granted a read-lock on X ✓

2. Why was T1 aborted?

(i) "Uncommitted data" problem. ✓

(ii) Timestamp = 8 ✓

(iii) Data that is not committed is a problem because, in this case, Transaction 2 wants access to item C while Transaction 1 is not done with it, i.e. has not committed yet. This means transaction 1 in Time 8 gained access to a temporary data item, and this causes integrity issues whenever there's a rollback

The transaction property that is violated is the **isolation property**, which states that data cannot be used by a 2nd transaction until the 1st transaction is done with the data(commits).

3.

[1] `FROM vehicle`
`JOIN constructor ON vehicle.conCode = constructor.conCode` ✓
`JOIN service ON vehicle.vin = service.vin` ✓
`WHERE conName = 'Ferrari'` ✓
`AND serviceType LIKE '%oil%'` ✓

Explanation:

The SQL query is now optimised because we are using “JOIN” statements instead of old-style joins. Using JOIN also optimises the filtering process in the WHERE clause. The equality operation is now the first to be performed, which narrows the dataset to only 2 records (since there are 2 Ferraris), and other operations, such as LIKE, can be applied to a smaller dataset, unlike before. ✓